

C Language Programming: Homework #10
Assigned on 12/26/2024(Thursday), Due on 01/13/2025 (Thursday)

1. Write a program that reads data from a file containing a datum called prefix in each line of the file called **routing_table**. Each prefix has the format of IP/length as following examples:
0.2.64.0/15
0.3.8.0/22
1.3.33.0
1.3.38.0/23
2. The first part a.b.c.d is a 32-bit IP address. So, you have to store the IP address into an unsigned variable. For prefix length, you can use an unsigned character. So, for all the prefixes, you can use an array of **struct prefix {unsigned ip; unsigned char len; struct prefix *next;}** to store all the prefixes. Assume prefix 0.0.0.0/0 does not exist.
3. Do the following,
 - (a) write a function *input(..)* to read all the prefixes from the input file
 - (b) print out the total number of prefixes in the input file,
 - (c) write a function *group_len(...)* to divide the prefixes into groups based on prefix length.
 - (d) write a function *length_distribution(..)* to compute the number of prefixes with prefix length i , for $i = 0$ to 32,
 - (e) For each non-empty group of length i , create a hash table of size 256 by using the most significant 8 bits of IP part of the prefixes. Using singly linked list to chain the prefixes together in each of 256 buckets of the hash table. In other words, the prefixes put into a bucket of the hash table must have the same MSB 8 bits of their IP part.
 - (f) Therefore, you have to write a function *prefix_insert()* to insert a prefix in a one-by-one fashion in the increasing order of the unsigned numbers of the prefixes. A file named **inserted_prefixes** contains some prefixes that will be inserted after the length groups and hash tables are built from the file **routing_table**.
 - (g) Also, you have to write a function *prefix_delete()* to delete a prefix. The prefixes to be deleted after inserting all the prefixes from files **routing_table** and **inserted_prefixes** are from a file called **deleted_prefixes**.
 - (h) you have to write a function *search()* by giving an IP address to report if the search is successful or failed. The IP addresses to be searched are contained in a file called **trace_file**.
 - (i) Finally, you have to report average numbers of clock cycles to perform a search, an insertion, or a deletion and draw three figures as follows. How to measure the search/insertion/deletion in cycles is illustrated in *clock.c*

